



ACTAI • WEEK 04 • LECTURE 02

Real Databases

SQL vs NoSQL — and rebuilding our inventory app on Firebase Firestore.



Concepts + a live build on a proper database

WHERE WE LEFT OFF

The Sheet was great — until it wasn't



A Google Sheet taught us the idea: rows = records, columns = fields, and apps can read & write data.

But it hit walls:

- ✗ Slows down with lots of data
- ✗ No real logins or privacy
- ✗ No fast searching / relationships
- ✗ Not built for many users at once

Databases come in two families



SQL

Relational. Neat tables with fixed columns and strong links between them. Think spreadsheets that talk to each other.



NoSQL

Flexible. Collections of free-form documents (like JSON). No fixed shape, easy to change, built to scale.

Neither is 'better' — they suit different jobs. Let's see each, then compare.

SQL: structured & related



Like a spreadsheet on steroids

- Data lives in tables (rows & columns)
- Fixed schema — you define columns first
- Tables link together (orders → customers)
- Great for: structured, related, exact data

LOOKS LIKE — the SQL language

```
CREATE TABLE inventory (  
  item TEXT,  
  quantity INT,  
  reorder_level INT  
);  
  
SELECT * FROM inventory  
WHERE quantity <= reorder_level;
```

Examples: [PostgreSQL](#) · [MySQL](#) · [Supabase \(hosted Postgres\)](#)

FAMILY 2

NoSQL: flexible documents



Collections of free-form documents

- Documents look like JSON (key: value)
- No fixed shape — add fields anytime
- Scales easily; many can sync in real-time
- Great for: flexible, fast-changing data

LOOKS LIKE — a document

```
inventory (collection)
└ document {
    item: "Tomatoes",
    quantity: 12,
    reorderLevel: 5
}
```

Examples: **MongoDB** · **Firebase Firestore (real-time)**

SIDE BY SIDE

SQL vs NoSQL

	SQL	NoSQL
Structure	Tables: rows & columns	Collections of JSON documents
Schema	Fixed — define columns first	Flexible — add fields anytime
Relationships	Strong, built-in links	Looser; data often nested
You ask with	SQL (SELECT ...)	Methods (.get(), .find())
Best for	Structured, related data	Flexible, fast-changing, scale
Examples	PostgreSQL, Supabase	MongoDB, Firestore

Rule of thumb: SQL when data is structured & related; NoSQL when it's flexible or you want to move fast.

The same data, three ways

FIRESTORE (NoSQL document)

```
// collection "inventory", one document:  
{ item: "Tomatoes", category: "Vegetables", quantity: 12, unit: "kg", reorderLevel: 5 }
```

SUPABASE (SQL table)

```
CREATE TABLE inventory (id, item, category, quantity, unit, reorder_level);  
-- one row:  Tomatoes | Vegetables | 12 | kg | 5
```

MONGODB + MONGOOSE (NoSQL, defined in code)

```
const Inventory = mongoose.model("Inventory", new mongoose.Schema({  
  item: String, category: String, quantity: Number, unit: String, reorderLevel: Number }));
```

THE PLAYERS

Three you'll hear about



Firestore

NoSQL • real-time

Google's real-time document DB. Free Spark plan: 50k reads & 20k writes a day, 1GB — no credit card.



Supabase

SQL • Postgres

Open-source. A real SQL database with a friendly UI, APIs and logins. Generous free tier.



MongoDB + Mongoose

NoSQL • documents

Popular document DB. Mongoose lets you define data 'models' in code. Free Atlas cluster to start.

OUR PRACTICAL PICK

Why Firebase Firestore



A real, cloud database that's free to start and syncs in real-time — perfect for learning.



Free, no card

Spark plan; generous daily limits



Real-time

changes appear instantly, everywhere



Beginner-friendly

just collections & documents



Built into tools

many vibe tools connect natively

Collections → documents → fields



inventory (collection)

a collection = a group of items



document { item: "Tomatoes", quantity: 12, unit: "kg", reorderLevel: 5 }



document { item: "Chicken", quantity: 8, unit: "kg", reorderLevel: 5 }

each document = one item, with its own fields

STEP BY STEP

Set up Firestore (free)

1. Go to console.firebase.google.com → Add project
2. Name it and create (skip Analytics if you like)
3. Build → Firestore Database → **Create database**
4. Start in **test mode**, pick a location
5. Project settings → add a **Web app** → copy the config
6. Keep that config — your app needs it



Free, no card

Spark plan: 50k reads & 20k writes per day, 1GB. Plenty for class.



'Test mode' note

Test mode is open to anyone — fine for learning. We lock it down with rules next lecture.

CONNECT IT

Tell your app to use Firestore

Build the app in your vibe-coding tool and hand it the Firebase config. Many tools (like AI Studio) can connect Firebase in one click.

TELL YOUR APP

Use Firebase Firestore as the database. Here is my config:

[PASTE YOUR FIREBASE CONFIG]

Store inventory items in a collection called "inventory".

Load them when the app opens, let me add and edit items, and update the list in real-time as the data changes.

Same pattern as always: the app talks to the database. Only the database changed — from a Sheet to Firestore.

WATCH THIS

Live build: inventory on Firestore

Same restaurant app as yesterday — now on a real, real-time database.

BUILD PROMPT — paste into your tool

Build a restaurant inventory manager using Firebase Firestore [config: PASTE]. Store items in an "inventory" collection with fields: item, category, quantity, unit, reorderLevel. Show them in a clean table, highlight rows in red when quantity <= reorderLevel, and let me add and edit items. Update the table in real-time when the data changes.

THEN TEST IT — the real-database magic

Add an item — it saves to Firestore and appears instantly.

Open the same app on your phone: the data is there too. Edit on one, watch it update on the other — live.

Limits & cost awareness



Free tier is generous, not infinite

50k reads / 20k writes a day — fine for class, watch at scale.



Reads cost money on Blaze

Big apps: every read counts. Set a budget alert.



NoSQL has no built-in joins

Structure data for how you'll read it; plan ahead.



Lock down 'test mode'

It's open to anyone right now — rules & logins next lecture.

Most importantly: test mode is open to the world. We fix that next lecture with rules + logins.



LECTURE RECAP

You speak database now.

SQL = structured tables; NoSQL = flexible documents. You rebuilt the inventory on Firestore — a real, real-time database.

Homework · move one of your apps onto Firestore. **Next** — authentication: real logins.